eBPF-XDP Hardware-Based Network Slicing for 6G Networks

Complete Flow and Implementation Plan

Contents

1 Paper Overview **3**

2 High-Level System Architecture **3**

3 Industry 4.0 Use Cases **3**
- 3.1 PLC Control Function Offloading to Edge . . . 3
- 3.2 Smart Transportation Vehicles (AGVs) . . . 3
- 3.3 Advanced Network Slicing . . . 4
- 3.4 Animal Tracking in Indoor Farming . . . 4
- 3.5 Airport Baggage Handling Robots . . . 4

4 Proposed Framework Architecture **4**
- 4.1 Layer 1: Hardware (SmartNIC – Agilio CX) . . . 4
- 4.2 Layer 2: Kernel Space (Linux Kernel 5.11+) . . . 5
- 4.3 Layer 3: User Space (Application Level) . . . 5

5 Data Flow Pipeline **5**
- 5.1 Packet Processing Flow (Algorithm 1) . . . 5

6 Slice Definition and Control **6**
- 6.1 Intent-Based Message Format (Listing 1) . . . 6
- 6.2 Packet Metadata Structure (Listing 2) . . . 7

7 Key Implementation Components **7**
- 7.1 eBPF Program (Offloaded in SmartNIC) . . . 7
- 7.2 Custom NFP Driver . . . 7
- 7.3 Control Application (User Space) . . . 8
- 7.4 Monitoring Application (User Space) . . . 8

8 Performance Results (Experimental Validation) **8**
- 8.1 Test Setup . . . 8
- 8.2 Key Metrics . . . 8
  - 8.2.1 CPU Load . . . 8
  - 8.2.2 Packet Loss . . . 8
  - 8.2.3 Bandwidth . . . 9
  - 8.2.4 Latency . . . 9

9 Implementation Phases **9**

10 Critical Implementation Details **10**
10.1 Memory and Performance Constraints 10
10.2 Software vs. Hardware Trade-offs 10
10.3 Challenges to Address 10

11 Deployment Architecture **10**

12 Technology Stack **11**

13 Prototype Environment for Demonstration **11**
13.1 Recommended Prototype Setup 11
13.2 What Can Be Demonstrated with Mininet 12
13.3 Prototype Limitations 12
13.4 Practical Conclusion 12

14 Next Steps for Implementation **13**

15 References for Deeper Understanding **13**

16 Paper Metadata **13**

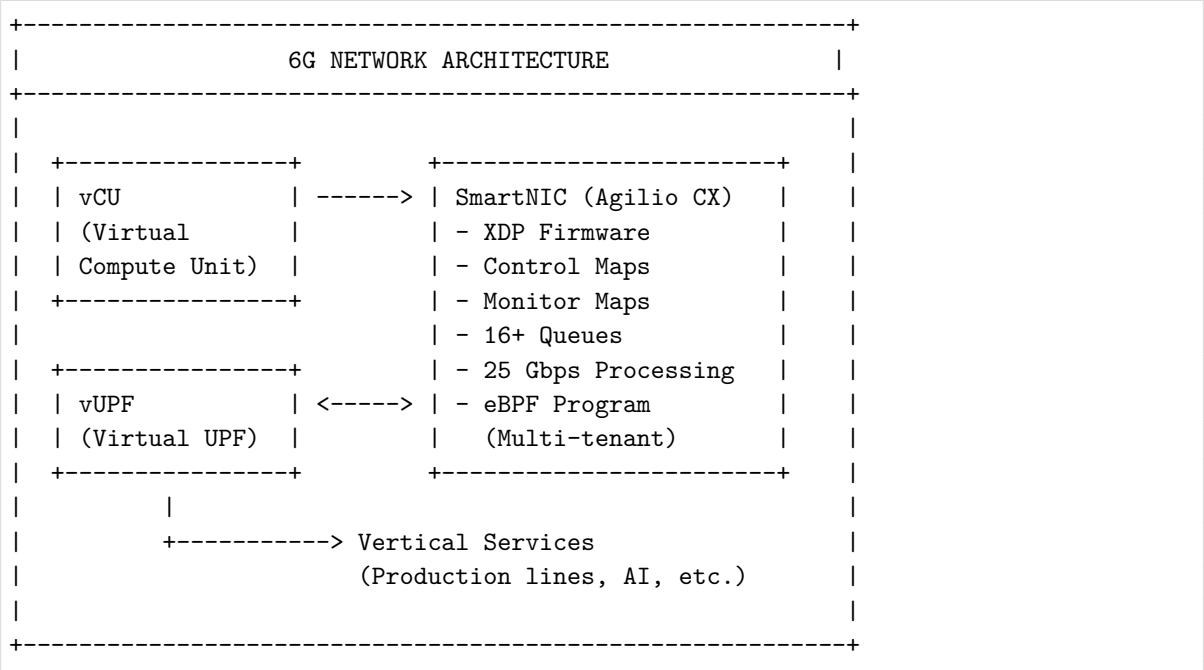
1 Paper Overview

Research Focus: Ultra-high-performance programmable data plane for future 6G networks using eBPF-XDP with hardware offloading on a SmartNIC.

Key Innovation: Combining hardware-offloaded eBPF/XDP with AF_XDP sockets (kernel bypass) to achieve:

- High-performance packet processing (25 Gbps)
- Dynamic network slicing
- Real-time monitoring and control
- Support for Industry 4.0 use cases

2 High-Level System Architecture



3 Industry 4.0 Use Cases

3.1 PLC Control Function Offloading to Edge

- Replaces fixed PLCs with virtualized controllers.
- Requirements: less than 1 second slice instantiation, low latency, high reliability.
- KPIs: latency below 10 ms, reliability above 99.9%.

3.2 Smart Transportation Vehicles (AGVs)

- Autonomous guided vehicles with video streaming (130 HD frames/sec, approximately 3 Gbps).

- Real-time object detection at the edge.
- KPIs: latency below 50 ms, data rate 3 Gbps, ultra-high precision.

3.3 Advanced Network Slicing

- Manages multiple traffic types with different SLAs.
- Supports flexible, on-demand network slices.
- Enables dynamic priority and bandwidth allocation.

3.4 Animal Tracking in Indoor Farming

- IoT monitoring with AI processing at the edge.
- Health and mobility detection for farm animals.
- KPIs: reliability above 99%, latency below 100 ms.

3.5 Airport Baggage Handling Robots

- Automated baggage sorting with video streams.
- Real-time AI detection and collision avoidance.
- KPIs: high reliability and low latency (below 50 ms).

4 Proposed Framework Architecture

4.1 Layer 1: Hardware (SmartNIC – Agilio CX)

```
+--------------------------------------+
| Netronome Agilio CX SmartNIC         |
+--------------------------------------+
| - NFP-4000 Processor                 |
| - 60 Flow Processing Cores           |
| - 48 Packet Processing Cores         |
| - 16 Hardware Queues (XSK sockets)   |
| - 2x SFP28 25GbE Interfaces          |
| - PCIe Gen3 2x8                      |
| - 25 Gbps Max Processing Rate        |
+--------------------------------------+
| CUSTOM XDP OFFLOADED FIRMWARE        |
|  1. Pre-6G Parser                    |
|     -> Extract metadata              |
|  2. Matching Module                  |
|     -> Hash lookup in maps           |
|  3. Action Executor                  |
|     -> Drop / Forward / Pass         |
|                                      |
| eBPF MAPS (2M rules each)            |
| - Control Map (32-bit key/value)     |
| - Monitoring Map (32-bit key,        |
```

```
|   64-bit value)                      |
+--------------------------------------+
```

4.2 Layer 2: Kernel Space (Linux Kernel 5.11+)

```
+--------------------------------------+
| Custom NFP Driver                    |
+--------------------------------------+
| - AF_XDP Support                     |
| - XSK Socket Management              |
| - Hardware Interrupt Handling        |
| - Memory Access Translation          |
| - Support for up to 16 sockets       |
+--------------------------------------+
```

4.3 Layer 3: User Space (Application Level)

```
+-------------------------------------------+
| Data Plane Programmability (DPP) Application |
+-------------------------------------------+
| - Control Application                     |
|   -> Insert / Remove rules                |
|   -> Hardware control map management      |
|                                           |
| - Monitoring Application                  |
|   -> Real-time statistics                 |
|   -> Monitoring map updates               |
|   -> Network metrics (PPS, size)          |
|                                           |
| - Virtual Services                        |
|   -> Vertical applications                |
|   -> Service logic                        |
|                                           |
| Libraries Used:                           |
| - libbpf                                  |
| - XSK                                     |
+-------------------------------------------+
```

5 Data Flow Pipeline

5.1 Packet Processing Flow (Algorithm 1)

```
INCOMING PACKET
    |
    v
1. PARSE HEADERS
   - Extract inner/outer MAC addresses
   - Extract inner/outer IP addresses
   - Extract inner/outer transport ports
```

```
   - Extract link and transport protocols
   - Extract VXLAN/GTP identifiers
    |
    v
2. CALCULATE 32-BIT HASH
   hash = hash(all_metadata)
    |
    v
3. LOOKUP CONTROL MAP
   control_entry = control_map.lookup(hash)
    |
    +-- NO MATCH ------> DEFAULT PASS (XDP_PASS)
    |
    v MATCH FOUND
4. CHECK ACTION (control_entry.action)
    |
    +-- ACTION 0 ------> DROP (XDP_DROP)
    |
    +-- ACTION 1 ------> STEER TO QUEUE
                         rx_queue <- control_entry.queue
                         Pass to XSK socket
    |
    v
5. UPDATE MONITORING MAP
   monitor_entry = monitor_map.lookup(hash)
   if exists: update(hash, pkt_length, ++pkt_count)
    |
    v
6. OUTPUT
   Forward to user space via AF_XDP socket
        |
        v
   Vertical service processing
        |
        v
   Update action in packet payload
        |
        v
   Return same socket to SmartNIC
        |
        v
   Forward back to origin via physical interface
```

6 Slice Definition and Control

6.1 Intent-Based Message Format (Listing 1)

```
{
  "service": "service_name",
  "resources": {
    "network_properties": [...]
  },
```

```
  "service_priority": integer,
  "MaxAllowedBandwidth": Gbps,
  "MinimumGuaranteedBandwidth": Gbps
}
```

6.2 Packet Metadata Structure (Listing 2)

```
struct pkt_meta {
  // MAC Layer
  uint8_t outer_src_mac[6];
  uint8_t outer_dst_mac[6];
  uint8_t inner_src_mac[6];
  uint8_t inner_dst_mac[6];

  // IP Layer
  uint32_t outer_src_ip;
  uint32_t outer_dst_ip;
  uint32_t inner_src_ip;
  uint32_t inner_dst_ip;

  // Transport Layer
  uint16_t outer_src_port;
  uint16_t outer_dst_port;
  uint16_t inner_src_port;
  uint16_t inner_dst_port;

  // Protocol and Tunneling
  uint8_t link_protocol;
  uint8_t transport_protocol;
  uint32_t vxlan_id;
  uint32_t gtp_id;
};
```

7 Key Implementation Components

7.1 eBPF Program (Offloaded in SmartNIC)

- **Language:** C
- **Execution:** Hardware pipeline (XDP firmware)
- **Purpose:** Packet parsing, hashing, rule matching, traffic steering
- **Maps:** Control map (2M rules), monitoring map (2M rules)

7.2 Custom NFP Driver

- **Purpose:** Bridge between the SmartNIC and Linux kernel
- **AF_XDP Support:** XSK socket integration
- **Max Queues:** 16 hardware queues
- **Customization:** Kernel bypass via XSK

7.3 Control Application (User Space)

- **Function:** Configure network rules
- **Operations:** Insert and delete entries in the control map
- **Input:** Intent-based messages with slice definitions

7.4 Monitoring Application (User Space)

- **Function:** Collect real-time metrics
- **Metrics:**
  - Packets per second (PPS) per socket
  - Packet count per rule
  - Packet length distribution
  - CPU usage per socket
  - Packet loss percentage
  - Round-trip time (RTT)

8 Performance Results (Experimental Validation)

8.1 Test Setup

- **Hardware:** Two CyberServe XE5-212S servers with Agilio CX 2x25GbE SmartNICs
- **OS:** Ubuntu 21.04, Linux kernel 5.11.0-40
- **Traffic:** Double-encapsulated RTP video (VXLAN/GTP)
- **Max Rate:** 25 Gbps, 20 million packets/sec
- **Packet Sizes:** 132 B to 1500 B
- **Control Rules:** Up to 131K rules in the hardware map

8.2 Key Metrics

8.2.1 CPU Load

- **132 B packets:** 1 socket uses approximately 70–100% CPU; 16 sockets use approximately 25% CPU.
- **1500 B packets:** 1 socket uses approximately 35% CPU; 16 sockets use approximately 5% CPU.

8.2.2 Packet Loss

- **Baseline (1 socket, 132 B):** approximately 70% loss.
- **4 or more sockets:** less than 1% loss.
- **Conclusion:** Traffic distribution across sockets is critical for reliability.

8.2.3 Bandwidth

- **Max throughput:** up to 20 Mpps (at 132 B).
- **Sweet spot:** approximately 5 Mpps per socket without packet loss.
- **Scalability:** roughly linear with socket count.
- **Formula:** BW(Gbps) = $\frac{\text{PPS} \times \text{PacketSize}}{10^9}$.

8.2.4 Latency

- **Critical KPI:** Pre-6G maximum latency = 0.1 ms (100 μs).
- **Achieved:** approximately 0.13 ms with 132 B packets at 25 Gbps.
- **At 1500 B:** approximately 0.55 ms, which exceeds the requirement.
- **Mitigation:** Distribute traffic across processes or servers for large packets.

9 Implementation Phases

1. **Phase 1: Foundation Setup**
   - Set up the SmartNIC environment (Agilio CX).
   - Install Linux kernel 5.11+ with eBPF support.
   - Compile the custom NFP driver with AF_XDP support.
   - Set up libbpf and XSK libraries.
2. **Phase 2: Core eBPF Program**
   - Develop packet parsing logic.
   - Implement 32-bit hash calculation.
   - Create control and monitoring maps.
   - Implement the three-component XDP firmware:
     - Pre-6G parser
     - Matching module
     - Action executor
3. **Phase 3: Control Plane**
   - Build the control application.
   - Add rule insertion, deletion, and slice lifecycle management.
   - Build the monitoring application.
   - Add real-time metric collection and performance analytics.
   - Implement the intent-based message handler.
4. **Phase 4: Integration and Testing**
   - Connect two servers (CU and vUPF).
   - Implement a traffic generator (Pktgen).

- Deploy vertical services in containers.
- Validate all five use cases.

5. **Phase 5: Performance Optimization**
   - Tune CPU affinity (`irq_affinity`).
   - Optimize socket distribution.
   - Optimize rule hashing.
   - Reduce latency for large packets.

10 Critical Implementation Details

10.1 Memory and Performance Constraints

- **Control Map:** 32-bit key, 32-bit value (16-bit action + 16-bit queue)
- **Monitoring Map:** 32-bit key, 64-bit value (32-bit length + 32-bit counter)
- **Lookup Complexity:** Constant-time $O(1)$ hardware map access
- **Max Rules:** 2M per map
- **XSK Sockets:** Up to 16 per SmartNIC

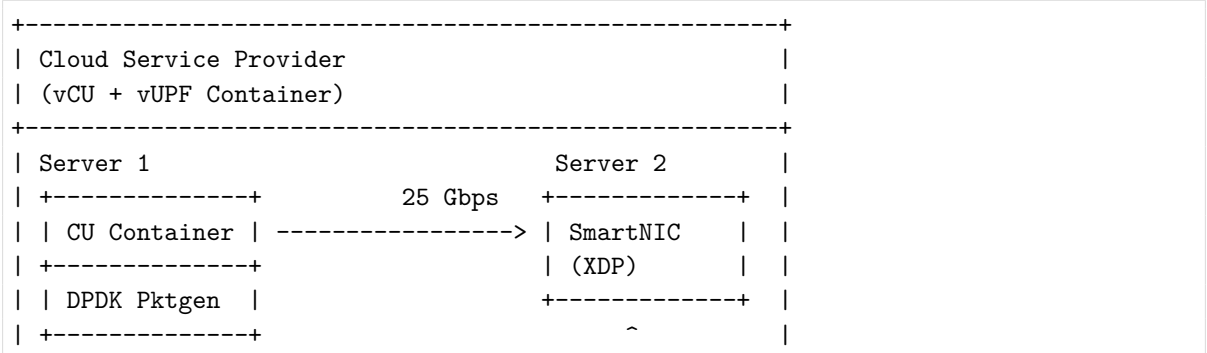
10.2 Software vs. Hardware Trade-offs

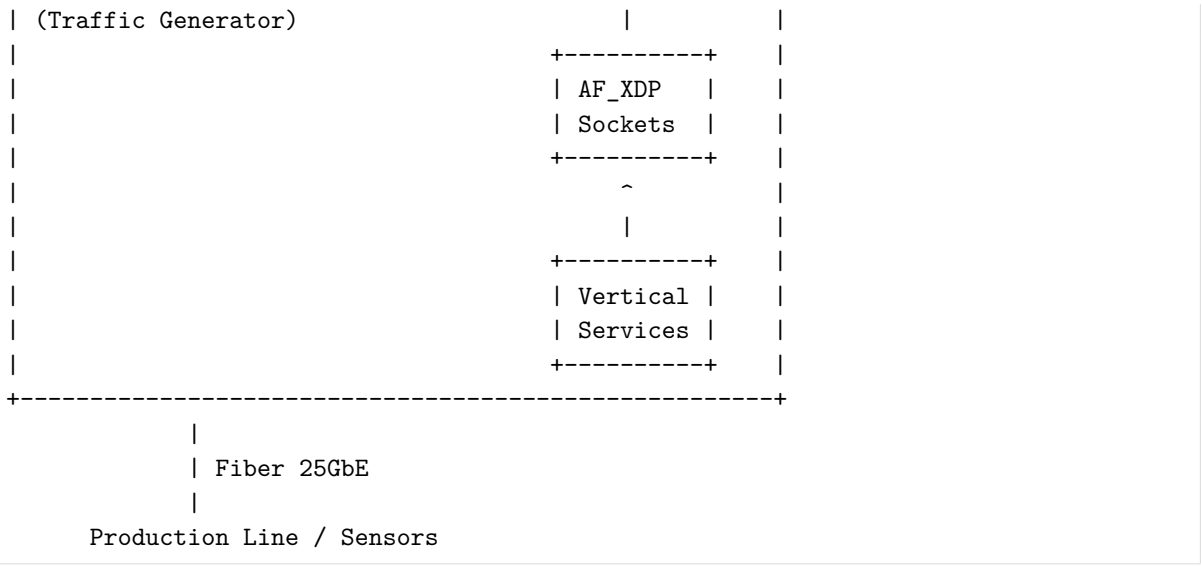
- **Hardware (XDP):** below 0.13 ms latency, 25 Gbps throughput, higher cost
- **Kernel (AF_XDP bypass):** reduced kernel overhead and faster user-space handling
- **Combined Approach:** strong performance-efficiency ratio

10.3 Challenges to Address

1. Large packet sizes (1500 B) exceed the 0.1 ms latency requirement.
2. A single socket becomes a bottleneck for small packets (132 B).
3. Complex network topologies need further study.
4. Geographic distance effects are not yet evaluated.

11 Deployment Architecture





12 Technology Stack

Component	Technology	Purpose
SmartNIC	Netronome Agilio CX	Hardware offloading
Packet Processing	eBPF + XDP	Ultra-low-latency forwarding
Kernel bypass	AF_XDP + XSK sockets	User-space acceleration
Driver	Custom NFP driver	SmartNIC-kernel bridge
eBPF Libraries	libbpf + XSK	Program and socket management
Traffic Generator	DPDK + Pktgen	Test traffic generation
Containerization	Linux containers	Vertical services
Protocol	Double-encapsulated VXLAN/GTP	Pre-6G traffic handling
OS	Ubuntu 21.04, Linux 5.11+	Runtime environment

13 Prototype Environment for Demonstration

13.1 Recommended Prototype Setup

For the current project stage, the best approach is a software-only prototype that can be demonstrated on a laptop or workstation using Linux and Mininet. The minimum setup can be:

- One laptop, desktop, or Linux workstation
- Multi-core CPU
- At least 8–16 GB RAM
- Standard Ethernet NIC or VM network interface

- Linux environment with eBPF/XDP support
- Mininet for network emulation and demonstration

This setup is sufficient for:

- eBPF/XDP packet parsing and classification
- Control-map and monitoring-map development
- Traffic-class or slice-policy demonstration
- User-space control and monitoring applications
- Emulated topologies using Mininet

Mininet is a good choice for this stage because it allows the slicing logic, rule management, and traffic differentiation behavior to be demonstrated without requiring SmartNIC hardware.

13.2 What Can Be Demonstrated with Mininet

- Multiple hosts generating different classes of traffic
- A virtual topology with switches and links
- eBPF/XDP-based packet filtering or classification
- Basic network slicing behavior through priority, forwarding, or drop policies
- Monitoring of packet counters, drops, and simple latency statistics
- Dynamic insertion and deletion of rules from user space

This is appropriate for:

- classroom or project demonstrations
- proof-of-concept validation
- early-stage development before hardware investment

13.3 Prototype Limitations

- Mininet does not reproduce SmartNIC hardware offload.
- Throughput and latency numbers will not match the paper's hardware results.
- AF_XDP and low-level NIC behavior may be limited in a VM-based setup.
- The prototype should be presented as a functional demonstration rather than a performance-equivalent replication.

13.4 Practical Conclusion

- For the best current prototype, a Linux-based software implementation with Mininet is fully acceptable.
- This approach is the most practical option for demonstration on existing hardware.
- Dedicated SmartNIC hardware can be considered later if the project moves toward full-scale validation.

14 Next Steps for Implementation

1. Procure the required hardware: Agilio CX SmartNICs and CyberServe servers.
2. Prepare the runtime environment: Ubuntu, kernel, and driver setup.
3. Implement the eBPF core: parser → hasher → matcher → executor.
4. Build the control plane: DPP application for orchestration.
5. Develop the test suite around the five Industry 4.0 use cases.
6. Optimize for large packets and geographic/topology constraints.
7. Deploy and validate in a production-like environment.

15 References for Deeper Understanding

- **eBPF:** Linux kernel extension for sandboxed programs
- **XDP:** eXpress Data Path for in-kernel NIC driver processing
- **AF_XDP:** Address family for kernel-bypass packet processing
- **VXLAN/GTP:** Tunneling protocols for network slicing
- **Network Slicing:** Logical network partition with dedicated resources
- **SmartNIC:** Programmable network interface card, such as Netronome Agilio CX

16 Paper Metadata

- **Paper Title:** *eBPF-XDP Hardware-Based Network Slicing Architecture for Future 6G Front- to Back-Haul Networks*
- **Authors:** Salva-Garcia et al.
- **Publication:** IEEE Transactions on Network and Service Management, Vol. 21, No. 2, April 2024